Java w IntelliJ IDEA - kompilacja i JVM

BLOK 1 - kompilacja, bajtkod, JVM i JIT

Punkt startowy: Po Lab 2 znasz już **klasę, metodę main, zmienne, typy i println**. Nie uczymy ich od nowa. Użyjemy znanego, krótkiego kodu do badania kompilacji, bajtkodu, JVM i JIT.

1. Co dzieje się między plikiem .java a wykonaniem?

Kod zapisany w pliku **.java** jest tekstem źródłowym. Kompilator **javac** przekształca go w instrukcje przeznaczone dla JVM, najczęściej w plikach **.class**. Dopiero uruchomienie programu powoduje wykonanie tych instrukcji. Bajtkod nie jest kodem maszynowym konkretnego procesora.

```
Plik .java  --(javac)-->  plik .class (bajtkod)
                                 |
                                 v
                           JVM: ładowanie klas
                                 |
                          interpretacja / JIT
                                 |
                                 v
                           działanie programu
```

Jeden plik źródłowy może doprowadzić do utworzenia kilku plików **.class**. Zmiana kodu źródłowego nie zmienia automatycznie wcześniej wyprodukowanego bajtkodu: wymagane jest ponowne zbudowanie programu.

2. JDK, javac, java i JVM — różne role

Element	Do czego służy
JDK	Zestaw narzędzi do programowania, m.in. javac, java i javap.
javac	Kompiluje pliki źródłowe .java do bajtkodu.
.class	Plik z bajtkodem przeznaczonym dla JVM.
java / JVM	Uruchamia program i wykonuje jego bajtkod.
javap	Pokazuje informacje o skompilowanej klasie, także instrukcje bajtkodu.

W terminalu można sprawdzić dostępne narzędzia. IntelliJ może korzystać z innego JDK niż terminal — obie konfiguracje należy sprawdzać osobno.

```
java -version
javac -version
javap -version
```

3. Kompilacja poza IntelliJ: pierwszy eksperyment

Otwórz **Demo.java**. Program jest celowo prosty, ponieważ celem ćwiczenia jest obserwacja narzędzi, a nie powtórka instrukcji println.

```
public class Demo {
    public static void main(String[] args) {
        System.out.println("Uruchomiono program");
    }
}
```

Uruchom poniższe polecenia **z katalogu projektu** (w którym znajduje się **src**). Pierwsze polecenie tworzy bajtkod; drugie uruchamia go z katalogu **out**.

```
javac -encoding UTF-8 -d out src/Demo.java
java -cp out Demo
```

Oczekiwany wynik: **Uruchomiono program**. Na komputerze bez katalogu **out** javac sam go utworzy. Przy pracy w IntelliJ odpowiednie czynności zwykle wykonuje mechanizm budowania projektu.

4. Co pokazuje bajtkod?

Otwórz **src/block1extra/CompilationStages.java**. Metoda **square** oblicza kwadrat liczby. Wydruk 49 nie wyjaśnia jeszcze, jakie instrukcje zapisano w pliku .class; do tego służy **javap -c**.

```
javac -encoding UTF-8 -d out src/block1extra/CompilationStages.java
javap -classpath out -c block1extra.CompilationStages
```

W wyniku szukaj instrukcji odpowiadających pobraniu argumentu, mnożeniu i zwróceniu wyniku (np. **iload**, **imul**, **ireturn**). Zmień w kodzie mnożenie na dodawanie, skompiluj ponownie i porównaj instrukcje; nazwy rozkazów nie zawsze odpowiadają pojedynczym liniom Javy.

Uwaga dotycząca pakietów: Jeśli plik ma deklarację **package block1extra;**, powinien znajdować się w **src/block1extra/**. Po kompilacji klasę uruchamiasz przez pełną nazwę: **java -cp out block1extra.CompilationStages**.

5. JVM, interpretacja i JIT

JVM ładuje potrzebne klasy, weryfikuje bajtkod i zarządza wykonywaniem oraz pamięcią. Implementacje JVM mogą interpretować instrukcje, a intensywnie wykonywane fragmenty kodu kompilować w trakcie działania do kodu maszynowego. Ten mechanizm nazywa się **JIT (just-in-time compilation)**.

JIT nie jest drugim kompilatorem, którego użytkownik musi ręcznie uruchamiać. Nie każdy fragment programu musi zostać skompilowany przez JIT. Krótki przykład uruchomiony tylko raz nie jest rzetelnym testem wydajności JIT.

6. Czy ten sam .class zadziała na innym komputerze?

Zazwyczaj tak, jeżeli komputer docelowy ma zgodną implementację JVM i odpowiednią wersję Javy oraz dostęp do potrzebnych bibliotek. Nie gwarantuje to jednak przenośności ścieżek plików ani funkcji specyficznych dla systemu. Bajtkod skompilowany dla nowszej wersji Javy może nie działać na starszej JVM.

```
javac --release 17 -encoding UTF-8 -d out src/Demo.java
```

Parametr **--release 17** określa poziom zgodności użytego języka i standardowego API oraz wersję generowanego bajtkodu. Projekty w tej paczce przygotowano do pracy z JDK 17 lub nowszym.

7. Rodzaje błędów — kiedy zostaną wykryte?

Sytuacja	Etap	Jak ją rozpoznać
Brak średnika	Kompilacja	javac nie tworzy poprawnej klasy; IntelliJ sygnalizuje błąd.
Dzielenie całkowite przez zero	Wykonanie	Program może zgłosić ArithmeticException.
Błędny wynik poprawnego kodu	Logika	Program działa, ale obliczenie jest niezgodne z zadaniem.

W **Demo.java** usuń na chwilę średnik po println. Odczytaj komunikat kompilatora, a następnie przywróć poprawny kod. Nie pozostawiaj celowego błędu w pliku przekazywanym do dalszych ćwiczeń.

8. Plan pracy z plikami projektu

Przykłady: Demo.java (javac/java), **CompilationStages.java** (bajtkod), **ToolchainExperiment.java** (narzędzia), **RuntimeEnvironment.java** (JVM), **Arguments.java** (argumenty startowe). **Zadania: BytecodeQuestions.java** i **PlatformReport.java**. Wszystkie pliki i ścieżki są wypisane w README projektu.

9. Jak zorganizowany jest katalog projektu?

Projekt otwierany w IntelliJ to nie to samo co pojedynczy plik Java. W tej paczce katalog **src** zawiera kod źródłowy. Podczas kompilowania powstają pliki **.class**, zwykle przechowywane poza katalogiem źródłowym. Ścieżki wygenerowanych klas muszą odpowiadać ich deklaracjom **package**. Dzięki temu środowisko uruchomieniowe odnajduje właściwe klasy.

```
Java_IntelliJ_material_temat_01/
    src/
        Demo.java
        block1extra/
            CompilationStages.java
    out/
        Demo.class
        block1extra/
            CompilationStages.class
```

Katalog **out** jest tu tylko przykładowym katalogiem wyjściowym kompilatora. Po otwarciu projektu IntelliJ może używać innego miejsca na wygenerowane klasy. Warto sprawdzić ustawienia budowania projektu zamiast zakładać, że wszystkie pliki **.class** znajdują się obok kodu źródłowego.

W pliku **CompilationStages.java** deklaracja pakietu brzmi **package block1extra;**. Jeśli przeniesiesz plik bez zmiany pakietu do katalogu **src/block1**, IntelliJ może wskazywać niezgodność. Najbezpieczniej zachować układ folderów otrzymany w paczce. Deklaracja pakietu nie jest ozdobą: stanowi część pełnej nazwy klasy.

10. Uruchamianie klasy i ścieżka wyszukiwania

Program **java** nie szuka pliku **.java** w bieżącym katalogu podczas klasycznego uruchamiania skompilowanego projektu. Otrzymuje nazwę klasy i listę miejsc, w których może jej szukać. Do określenia takich miejsc służy tzw. **classpath**, ustawiany między innymi opcją **-cp**.

```
javac --release 17 -encoding UTF-8 -d out src/block1extra/CompilationStages.java
java -cp out block1extra.CompilationStages
```

W drugim poleceniu wpisujemy **block1extra.CompilationStages**, a nie nazwę pliku zakończoną **.java** lub **.class**. Jeżeli użyjesz **java -cp out CompilationStages**, program może zgłosić błąd znalezienia klasy, chociaż jej bajtkod istnieje. Powodem jest pominięcie pełnej nazwy pakietu.

Przećwicz dwie sytuacje: najpierw uruchom poprawną komendę, a potem celowo usuń nazwę pakietu. Zapisz treść komunikatu i wskaż, co dokładnie zmieniło się w sposobie wyszukiwania klasy. Nie modyfikuj przy tym kodu programu.

11. Bajtkod nie jest prostym przepisaniem źródła

Kompilator musi zamienić instrukcje Javy na operacje wykonywane przez maszynę wirtualną. Dla bardzo prostej metody obliczającej kwadrat argumentu możesz zobaczyć instrukcje ładowania wartości, mnożenia i zwrotu wyniku. Liczba instrukcji bajtkodu nie musi być równa liczbie wierszy w źródle. Nie wszystkie szczegóły wysokopoziomowego zapisu są w bajtkodzie widoczne w tej samej postaci.

```
static int square(int x) {
    return x * x;
}

// W uproszczonym odczycie javap -c możesz zobaczyć:
// iload_0, iload_0, imul, ireturn
```

W **javap -c** zwykle pojawia się również numer pozycji instrukcji. Przeanalizuj odczyt bez próby ręcznej edycji pliku **.class**. Zmień w źródle **x * x** na **x + x**, skompiluj ponownie i sprawdź, który rozkaz bajtkodu uległ zmianie. Wynik programu także się zmieni. Następnie przywróć mnożenie.

Przykład pokazuje odrębność trzech etapów: **edycji źródła**, **kompilacji bajtkodu** i **wykonania**. Jeżeli po zmianie **.java** uruchomisz poprzedni plik **.class** bez ponownej kompilacji, nadal możesz otrzymać stary wynik. To ważne przy diagnozowaniu pozornie niewytłumaczalnych rezultatów.

12. Rozpoznawanie problemów podczas pracy

Objaw	Co sprawdzić
Błąd kompilatora przy średniku lub nawiasie	Czy kod źródłowy jest poprawny składniowo?
Could not find or load main class	Czy wpisano pełną nazwę klasy i poprawny classpath?
UnsupportedClassVersionError	Czy JVM nie jest starsza od wersji bajtkodu?
IntelliJ uruchamia stary wynik	Czy zapisano plik i wykonano nową kompilację?
Różna wersja Javy w terminalu i w IntelliJ	Czy terminal i projekt korzystają z tego samego JDK?

W komunikacie błędu zwracaj uwagę na nazwę klasy, ścieżkę i etap działania. Komunikat o braku klasy nie musi oznaczać błędnego kodu Java; może wskazywać na niepoprawny katalog roboczy lub sposób uruchomienia. Analogicznie poprawna kompilacja nie dowodzi, że program działa zgodnie z wymaganiami.

Eksperyment końcowy: porównaj uruchomienie tego samego **CompilationStages** z IntelliJ i z terminala. W obu przypadkach zapisz: użyte JDK, katalog kompilacji, polecenie lub konfigurację uruchomienia i otrzymany wynik. Jeżeli wyniki się różnią, sprawdź najpierw konfigurację, a dopiero potem kod.

13. Argumenty wiersza poleceń a metoda main

Z Lab 2 pamiętasz zapis **main(String[] args)**. W tym miejscu interesuje nas nie składnia metody, lecz sposób przekazywania argumentów podczas uruchamiania. Wartości wpisane za nazwą klasy trafiają do tablicy args jako napisy. Liczba argumentów zależy więc od wywołania, a nie od kompilatora.

```
public class Arguments {
    public static void main(String[] args) {
        System.out.println("Liczba argumentów: " + args.length);
    }
}
```

```
javac --release 17 -d out src/Arguments.java
java -cp out Arguments
java -cp out Arguments Ala 20 Warszawa
```

Pierwsze uruchomienie wypisze **Liczba argumentów: 0**, a drugie **Liczba argumentów: 3**. W IntelliJ analogiczny efekt można uzyskać, wpisując argumenty programu w konfiguracji uruchamiania. Nie należy ich mylić z opcjami samej JVM wpisywanymi przed nazwą klasy.

14. Interpretacja i JIT — co można zaobserwować?

Sama informacja, że JVM wykorzystuje JIT, nie oznacza, że łatwo zobaczyć moment kompilacji każdego fragmentu kodu. Maszyna wirtualna może analizować częstotliwość wywołań metod i optymalizować ich wykonanie w trakcie pracy programu. Z tego powodu krótkie programy szkoleniowe nie są wiarygodnym materiałem do porównywania wydajności interpretacji i JIT.

W wielu instalacjach OpenJDK/HotSpot można wymusić tryb interpretacji, stosując opcję **-Xint**. Poniższy program powinien wciąż wypisać 49, ponieważ zmiana trybu wykonania nie zmienia logiki obliczeń. Opcja ta jest zależna od implementacji JVM, dlatego nie należy zakładać jej dostępności w każdej maszynie wirtualnej.

```
java -cp out block1extra.CompilationStages
java -Xint -cp out block1extra.CompilationStages
```

Nie oceniaj pracy JIT tylko na podstawie czasu wyświetlonego przez IntelliJ przy pojedynczym uruchomieniu. Czas startu JVM, ładowanie klas i działanie środowiska mogą mieć większy wpływ na wynik niż samo mnożenie dwóch liczb.

15. Sprawdź, co zależy od komputera

Klasa **RuntimeEnvironment** pobiera z właściwości systemowych informacje o wersji Javy, dostawcy maszyny wirtualnej, nazwie systemu i architekturze. Podobne właściwości możesz wykorzystać w zadaniu **PlatformReport**. Są to wartości dostarczane przez środowisko uruchomieniowe, dlatego nie należy wpisywać w PDF jednego sztywnego, obowiązkowego wyniku.

```
System.getProperty("java.version")
System.getProperty("java.vendor")
System.getProperty("os.name")
System.getProperty("os.arch")
```

Porównaj wartości z wynikami poleceń **java -version** i **javac -version**. Jeżeli wskazują różne wersje, sprawdź ustawienia SDK projektu w IntelliJ oraz zmienne środowiskowe używane przez terminal. Wniosek nie powinien brzmieć jedynie „komputery są różne”, lecz wskazywać, które narzędzie i które ustawienie odpowiada za daną informację.

16. Krótka procedura diagnostyczna

Pracując nad kolejnymi tematami, stosuj stały sposób rozpoznawania problemów. Najpierw odróżnij błąd składni od błędu uruchamiania. Następnie sprawdź, czy problem dotyczy lokalizacji klasy, konfiguracji JDK, czy samej logiki obliczeń. Ten schemat pomoże także podczas późniejszych ćwiczeń z wyjątkami i plikami.

Etap kontroli	Co zapisać w notatkach
Kod źródłowy	Nazwa pliku, deklaracja package, metoda main.
Kompilacja	Użyty javac/JDK i ewentualny komunikat błędu.
Uruchomienie	Nazwa klasy, classpath i argumenty.
Wynik	Wypisane dane oraz ich zgodność z oczekiwaniem.
Wniosek	Czy problem dotyczył kodu, środowiska czy wykonania?

Przykłady źródłowe do analizy

Poniższe klasy pochodzą z projektu IntelliJ i są przeznaczone do **kompilowania, uruchamiania oraz porównywania bajtkodu**. Podane fragmenty to kompletne klasy, a nie nowe zadania o zmiennych.

CompilationStages.java

Eksperyment z metodą square i instrukcją **imul**.

```
package block1extra;

public class CompilationStages {
    static int square(int x) {
        return x * x;
    }

    public static void main(String[] args) {
        int result = square(7);
        System.out.println(result);
    }
}
```

Wynik: 49

ToolchainExperiment.java

Ten sam program można zbudować w IDE i poleceniem javac.

```
package block1extra;

public class ToolchainExperiment {
    public static void main(String[] args) {
        int a = 7;
        int b = 5;
        int result = a + b;
        System.out.println("Wynik: " + result);
    }
}
```

Wynik: 12

RuntimeEnvironment.java

Wynik zależy od wersji JDK oraz systemu komputera.

```
package block1extra;

public class RuntimeEnvironment {
    public static void main(String[] args) {
        System.out.println("Java version: " + System.getProperty("java.version"));
        System.out.println("Java vendor: " + System.getProperty("java.vendor"));
        System.out.println("OS: " + System.getProperty("os.name"));
        System.out.println("Architecture: " + System.getProperty("os.arch"));
    }
}
```

Wynik: wersja JVM, dostawca i system zależą od środowiska

17. Ćwiczenia do wykonania

Zadanie 1. Dla **CompilationStages** przewidź wynik **square(7)**. Następnie skompiluj klasę i wskaż w bajtkodzie instrukcję mnożenia. Zmień działanie na dodawanie i porównaj wynik **javap -c**.

Zadanie 2. Uruchom **RuntimeEnvironment** i zapisz informacje o JVM oraz systemie. Które mogą być inne na komputerze drugiego studenta i dlaczego?

Zadanie 3. W **Demo.java** celowo usuń średnik, zapisz komunikat kompilatora i napraw kod. Wyjaśnij, dlaczego program nie dochodzi do etapu wykonywania.

Zadanie 4. Z katalogu projektu porównaj: javac -d out src/Demo.java i
java -cp out Demo. Które polecenie kompiluje, a które uruchamia klasę?

Zadanie 5. Wyjaśnij, od czego zależy przenośność pliku **.class** i dlaczego bezwzględna ścieżka **C:\dane.txt** może nie działać na innym komputerze.